LPU ENGINEERING OPEN ▪ MASTER TEXTBOOK SERIES

University Curriculum ▪ Electronics & Computer Engineering

Introduction to Python

Comprehensive Engineering Textbook (ECE181)

Course Code: ECE181 ▪ **Credit Distribution:** 3-0-2 (4 Credits)
Curriculum: Basic Python, NumPy Vectorization, Pandas & Data Visualization
Target Audience: B.Tech Electronics and Computer Engineering Undergraduates

Authoritative Pedagogical Coverage:

- Unit 1: Introduction, Variables, Operators, Expressions & I/O
- Unit 2: Conditionals, Iterations, Functions & Recursion
- Unit 3: Strings, Lists, Tuples, Dictionaries & Sets
- Unit 4: Vectorized Computing & Matrix Operations with NumPy
- Unit 5: Data Analysis with Pandas & Visualization with Matplotlib
- Unit 6: Object-Oriented Design, File I/O, Exceptions & Applications

Course Director & Faculty Board:
LPU School of Electronics and Electrical Engineering
Department of Embedded Systems and Computational Engineering

Independent Academic Editorial Initiative ▪ 2026–27 Edition
Open Educational License (CC BY-NC-SA 4.0)

ECE181: Introduction to Python
Master University Textbook & Practical Engineering Manual

Published by: LPU Engineering Open Initiative
Edition: 2026–2027 Academic Year Edition
Primary Curriculum Code: ECE181 (Introduction to Python)

Copyright & Intellectual Property:
© 2026 The Faculty and Academic Editorial Board of LPU Engineering Open.
This work is released under the **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** license.
All source code listings and computational benchmarks are distributed under the **MIT License**.

Academic Advisory Statement:
This publication is authored in accordance with the official course syllabus of Lovely Professional University for ECE181. It serves as both an instructional textbook and an authoritative reference manual for scientific computing, vectorized numerical modeling, and data pipelines.

Pedagogical Architecture & How to Use This Book

This textbook is structured around the six rigorous units of the ECE181 curriculum:

1. **Formal Computational Principles:** Every concept begins with formal semantics, memory references, CPython execution models, and asymptotic time complexities.
2. **Clean Idiomatic Code:** High-yield code examples illustrate idiomatic Python 3 standards following PEP 8 conventions.
3. **Step-by-Step Worked Problems:** Practical algorithms with execution traces, variable state transitions, and memory layout diagrams.
4. **Examiner Traps & Edge Cases:** Callout warnings highlighting standard examination traps, operator precedence gotchas, and indexing subtleties.

Contents

1 Introduction to Python: Variables, Expressions, and Statements **1**
- 1.1 Introduction to the Python Programming Language . . . 1
  - 1.1.1 Interactive Mode vs. Script Mode . . . 1
- 1.2 Identifiers, Variables, and Memory Reference Model . . . 1
  - 1.2.1 Identifier Naming Rules and Conventions (PEP 8) . . . 2
- 1.3 Core Built-In Data Types . . . 2
- 1.4 Operators, Precedence, and Expression Evaluation . . . 2
  - 1.4.1 Arithmetic Operators . . . 3
  - 1.4.2 Relational and Bitwise Operators . . . 3
- 1.5 Standard Input and Formatted Output . . . 3
- 1.6 Chapter Summary and Review Questions . . . 4

2 Conditionals, Iterations, Functions, and Recursion **5**
- 2.1 Control Flow and Conditional Branching . . . 5
  - 2.1.1 Syntax Structures: `if`, `if-else`, and `if-elif-else` . . . 5
  - 2.1.2 Short-Circuit Evaluation . . . 5
- 2.2 Iterative Control Structures . . . 5
  - 2.2.1 The `while` Loop and Sentinel Controls . . . 5
  - 2.2.2 The `for` Loop and the `range()` Sequence Generator . . . 6
- 2.3 Modular Architecture: Functions . . . 6
  - 2.3.1 Parameter Binding Semantics . . . 6
  - 2.3.2 Scope Resolution: The LEGB Rule . . . 7
- 2.4 Functional Paradigms and Anonymous Functions . . . 7
- 2.5 Recursion and Memory Stack Mechanics . . . 7
- 2.6 Chapter Summary and Review Questions . . . 8

3 Data Structures: Strings, Lists, Tuples, Dictionaries, and Sets **9**
- 3.1 Sequences: Strings and Lists . . . 9
  - 3.1.1 Strings: Immutable Text Sequences . . . 9
  - 3.1.2 Lists: Dynamic Heterogeneous Arrays . . . 9
  - 3.1.3 List Comprehensions . . . 9
- 3.2 Tuples: Immutable Heterogeneous Records . . . 10
- 3.3 Dictionaries: Key-Value Hash Maps . . . 10
  - 3.3.1 CPython Hash Table Architecture . . . 10
- 3.4 Sets: Mathematical Collection of Unique Elements . . . 11
- 3.5 Chapter Summary and Review Questions . . . 11

4 Scientific Computing with NumPy: Vectorized Arrays **12**
- 4.1 Motivation: Why NumPy for Engineering? . . . 12
- 4.2 Array Creation and Structural Inspection . . . 12
- 4.3 Indexing, Slicing, and Memory Views vs. Copies . . . 13
- 4.4 Vectorization and Universal Functions (ufuncs) . . . 13
- 4.5 Linear Algebra Operations . . . 13
- 4.6 Chapter Summary and Review Questions . . . 14

5 Data Analysis with Pandas and Visualisation with Matplotlib 15
5.1 Introduction to Tabular Data Analysis with Pandas 15
5.1.1 Primary Data Structures: Series and DataFrame 15
5.1.2 Selection and Indexing: `.loc[]` vs. `.iloc[]` 15
5.1.3 Data Cleaning: Handling Missing Values 16
5.1.4 Aggregation and Grouping: Split-Apply-Combine 16
5.2 Engineering Data Visualisation with Matplotlib 16
5.2.1 Matplotlib Object-Oriented Hierarchy 16
5.3 Standard Statistical Plot Types 17
5.4 Chapter Summary and Review Questions 17

6 Real-World Engineering Applications: OOP, File I/O, and Exceptions 18
6.1 Object-Oriented Programming (OOP) in Python 18
6.1.1 Classes, Instances, and the `self` Parameter 18
6.1.2 Inheritance and Polymorphism 18
6.2 Defensive File Handling and Context Managers 19
6.2.1 The `with` Statement (Context Manager Protocol) 19
6.3 Robust Exception Handling 19
6.4 Real-World Case Study: Automated Diagnostic Log Processing 20
6.5 Chapter Summary and Review Questions 20

Chapter 1

Introduction to Python: Variables, Expressions, and Statements

1.1 Introduction to the Python Programming Language

Python is a high-level, interpreted, dynamically typed, multi-paradigm programming language created by Guido van Rossum and first released in 1991. In modern engineering, scientific computing, and embedded electronics, Python serves as the lingua franca for data acquisition, algorithm prototyping, hardware interfacing, and machine learning pipelines.

Definition: CPython Architecture and Execution Pipeline

Python programs are not directly translated into host machine microcode. Instead, the standard reference implementation, **CPython**, executes source code through a two-stage virtual machine pipeline:

1. **Compilation Phase:** The Python parser parses source code (`.py`) into an Abstract Syntax Tree (AST), checks syntax, and compiles statements into platform-independent intermediate instructions known as **bytecode** (typically serialized as `.pyc` files inside the `__pycache__` cache directory).
2. **Interpretation Phase:** The **Python Virtual Machine (PVM)** iteratively evaluates the bytecode stream, translating bytecode opcodes into host operating system system calls and native CPU operations.

1.1.1 Interactive Mode vs. Script Mode

Python provides two distinct execution execution paradigms:

- **Interactive Mode (REPL):** Executed via the command-line prompt (`>>>`) or Jupyter notebooks, providing an immediate Read-Eval-Print Loop. It is optimal for experimenting with expressions, inspecting hardware registers, and rapid testing.
- **Script Mode:** Executing stored instructions written in source files (`python script.py`). This is required for production engineering systems, automated cron tasks, and embedded daemon scripts.

1.2 Identifiers, Variables, and Memory Reference Model

In C and C++, a variable is a named storage address in memory whose bit width is determined by its static type. In contrast, Python implements a purely **object-reference model**.

Principle: Python Object Reference Model

In Python:

1. **Everything is an object:** Primitive numbers, strings, functions, classes, and modules

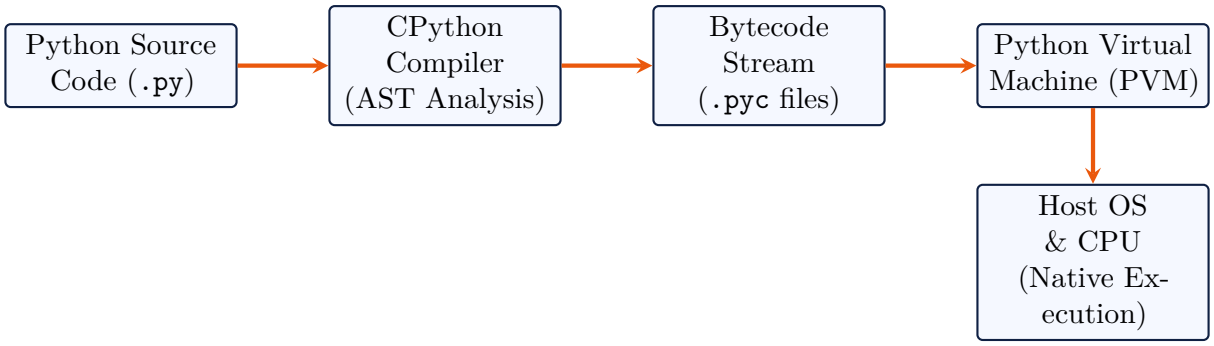


Figure 1.1: CPython Two-Stage Compilation and Virtual Machine Execution Pipeline.

are heap-allocated objects containing metadata headers (type pointer, reference count).

2. **Variables are reference pointers:** A variable name is merely an identifier bound to an object in the local or global namespace dictionary.
3. **Dynamic Typing:** Variable bindings carry no fixed data type constraint; the type resides entirely within the heap object itself.

```
# Variable binding and identity
a = 256
b = 256
print(a is b)       # True: CPython small integer caching pool [-5, 256]

x = 1000
y = 1000
print(x == y)       # True: Value equality check
print(x is y)       # May be False: Distinct heap objects outside cache
print(id(x), id(y)) # Inspect unique memory address identifiers
```

Listing 1.1: Demonstrating Object Identity and References in Python

1.2.1 Identifier Naming Rules and Conventions (PEP 8)

- Identifiers must begin with an alphabetic character (`a-z`, `A-Z`) or an underscore (`_`).
- Subsequent characters can be alphanumeric or underscores. Punctuation and symbols (`@`, `$`, `%`) are strictly invalid.
- Identifiers are strictly case-sensitive (`Voltage`, `voltage`, and `VOLTAGE` represent three distinct namespaces).
- Keywords (such as `def`, `class`, `yield`, `lambda`, `finally`) cannot be used as variable names.

1.3 Core Built-In Data Types

Python provides five foundational atomic scalar data types:

1.4 Operators, Precedence, and Expression Evaluation

Python expressions are constructed from operands and operators. Operators are categorized by arity and semantic function.

Data Type	Class Name	Description and Semantics	Mutability
Integer	`int`	Arbitrary-precision signed integers (unbounded size)	Immutable
Floating-Point	`float`	IEEE 754 double-precision 64-bit floating point	Immutable
Complex	`complex`	Complex numbers with real and imaginary float parts	Immutable
Boolean	`bool`	Logical truth values (`True`, `False`), subtype of `int`	Immutable
String	`str`	Unicode character sequences	Immutable

Table 1.1: Foundational Built-In Scalar Data Types in Python.

1.4.1 Arithmetic Operators

- **Addition (`+`) and Subtraction (`-`):** Standard algebraic addition and negation.
- **Multiplication (`*`):** Performs scalar multiplication or sequence repetition.
- **True Division (`/`):** Always returns a `float`, e.g., `7 / 2` evaluates to `3.5`.
- **Floor Division (`//`):** Truncates fractional parts toward negative infinity, e.g., `7 // 2` evaluates to `3`, but `-7 // 2` evaluates to `-4`.
- **Modulo (`%`):** Computes the remainder satisfying $a = (a//b) \times b + (a\%b)$.
- **Exponentiation (`**`):** Evaluates power expressions with right-to-left associativity.

Trap: Right-Associativity of Exponentiation

Unlike standard left-associative arithmetic operators ($+$, $-$, $*$, $/$, $//$), the exponentiation operator (`**`) binds from **right to left**:

$$2 * *3 * *2 = 2 * *(3 * *2) = 2 * *9 = 512 \quad (\text{NOT } (2 * *3) * *2 = 8 * *2 = 64) \tag{1.1}$$

1.4.2 Relational and Bitwise Operators

Relational operators (`<`, `<=`, `>`, `>=`, `==`, `!=`) compare operands and return a `bool`. Python uniquely supports **chained comparisons**:

$$a < b < c \iff (a < b) \textbf{ and } (b < c) \tag{1.2}$$

Bitwise operators manipulate raw bit-patterns of integers:

- Bitwise AND (`&`): True if both bits are 1.
- Bitwise OR (`|`): True if either bit is 1.
- Bitwise XOR (`^`): True if exactly one bit is 1.
- Bitwise Inversion (`~`): Inverts all bits, satisfying `~x == -(x + 1)`.
- Left Shift (`«`) and Right Shift (`»`): Multiplies or divides integers by powers of 2.

1.5 Standard Input and Formatted Output

Interactive user input is received via the built-in `input(prompt)` function, which always returns a `str`. Type casting must be explicitly applied.

```
# Data acquisition input casting
voltage = float(input("Enter measured node voltage (V): "))
current = float(input("Enter loop branch current (A): "))

# Calculation
power = voltage * current
resistance = voltage / current if current != 0 else float('inf')

# Modern f-string formatted display with precision specifications
print(f"Power Dissipation : {power:10.4f} W")
print(f"Branch Resistance : {resistance:10.2f} Ohms")
print(f"Scientific Notation: {power:.3e} W")
```

Listing 1.2: Engineering I/O and Modern String Formatting (f-strings)

1.6 Chapter Summary and Review Questions

1. Explain the two-step execution model of CPython. How does bytecode differ from machine code?
2. What is small integer caching, and why does `a is b` evaluate to `True` for `a=200, b=200` but can be `False` for `a=1000, b=1000`?
3. Evaluate the exact expression: `result = 4 + 3 * 2 ** 3 // 4 - 5 % 2`.
4. Write a script that reads temperature in Fahrenheit from the console and outputs Celsius and Kelvin using `f-strings` formatted to 2 decimal places.

Chapter 2

Conditionals, Iterations, Functions, and Recursion

2.1 Control Flow and Conditional Branching

Programs make decisions by evaluating boolean expressions to alter the sequence of instruction execution. Python uses strict indentation blocks (conventionally 4 spaces) rather than curly braces to define scope boundaries.

2.1.1 Syntax Structures: `if`, `if-else`, and `if-elif-else`

```
signal_level_dbm = -45.0

if signal_level_dbm >= -30.0:
    quality = "Strong Signal (Optimal)"
elif signal_level_dbm >= -60.0:
    quality = "Good Signal (Acceptable)"
elif signal_level_dbm >= -80.0:
    quality = "Weak Signal (Marginal)"
else:
    quality = "No Signal / Link Failure"

print(f"Receiver Status: {quality}")
```

Listing 2.1: Multi-way Signal Classification

2.1.2 Short-Circuit Evaluation

Python evaluates logical operators (`and`, `or`) using short-circuit semantics:

- **`A and B`:** If A is false, Python returns A immediately without evaluating B.
- **`A or B`:** If A is true, Python returns A immediately without evaluating B.

This allows safe guard expressions, such as `if denom != 0 and (num / denom) > threshold:`.

2.2 Iterative Control Structures

2.2.1 The `while` Loop and Sentinel Controls

The `while` statement executes a suite of instructions repeatedly as long as a test condition remains true. It is well-suited for iterative numerical approximations (e.g., Newton-Raphson method) where the iteration count is not known a priori.

```
def newton_sqrt(n, tolerance=1e-7):
    if n < 0:
        raise ValueError("Cannot compute real square root of negative
            number.")
    if n == 0:
```

```
        return 0.0
    x = n / 2.0
    while True:
        root_next = 0.5 * (x + n / x)
        if abs(root_next - x) < tolerance:
            return root_next
        x = root_next

print("Calculated:", newton_sqrt(2.0))
```

Listing 2.2: Newton-Raphson Square Root Approximation

2.2.2 The `for` Loop and the `range()` Sequence Generator

Python `for` loops iterate over elements of any sequence or iterable protocol:

$$\texttt{range(start, stop, step)} \implies [start, start + step, start + 2 \cdot step, \cdots < stop] \tag{2.1}$$

Python supports loop control statements: `break` (terminates loop), `continue` (skips current iteration), and an optional `else` block that executes if and only if the loop terminates normally without encountering a `break`.

2.3 Modular Architecture: Functions

Functions encapsulate logic, promote reusability, and establish isolated local scopes.

2.3.1 Parameter Binding Semantics

Python functions support diverse parameter passing mechanisms:

1. **Positional Arguments:** Matched strictly left-to-right.
2. **Keyword Arguments:** Specified by name, independent of argument order (`func(b=2, a=1)`).
3. **Default Parameters:** Fallback values evaluated once at definition time.
4. **Arbitrary Positional Arguments (`*args`):** Bundles excess positional arguments into a `tuple`.
5. **Arbitrary Keyword Arguments (`**kwargs`):** Bundles excess keyword arguments into a `dict`.

Trap: Mutable Default Arguments Pitfall

Default arguments are evaluated once when the function definition is executed, **not** at invocation time. Never use mutable objects (like lists or dictionaries) as default arguments:

```
# BUG: The list accumulator persists across invocations!
def append_reading(val, readings=[]):
    readings.append(val)
    return readings

# CORRECT IDIOM:
def append_reading_safe(val, readings=None):
    if readings is None:
        readings = []
    readings.append(val)
    return readings
```

2.3.2 Scope Resolution: The LEGB Rule

Identifier lookup follows four nested namespaces:

1. **L (Local):** Names assigned inside current function.
2. **E (Enclosing):** Names in the enclosing functions (closures).
3. **G (Global):** Names assigned at the top-level of module file.
4. **B (Built-in):** Names pre-assigned in Python's `builtins` module.

Modifying variables outside local scope requires the `global` or `nonlocal` keywords.

2.4 Functional Paradigms and Anonymous Functions

Python provides `lambda` expressions for lightweight, anonymous single-expression functions:

```
from functools import reduce

voltages = [1.2, 3.3, 5.0, 12.0, 24.0]

# map: transform each element
scaled = list(map(lambda v: v * 1.05, voltages))

# filter: extract subset matching predicate
high_voltages = list(filter(lambda v: v >= 5.0, voltages))

# reduce: combine elements recursively
total_energy = reduce(lambda acc, v: acc + v**2, voltages, 0)
```

Listing 2.3: Higher-Order Functional Tools

2.5 Recursion and Memory Stack Mechanics

A recursive function calls itself to solve smaller subproblems until reaching a base condition.

Definition: Call Stack Frame Allocation

Every recursive call pushes an activation record (stack frame) containing local variables, parameters, and return program counters onto the execution call stack. If recursion depth exceeds `sys.getrecursionlimit()` (typically 1000), a `RecursionError` is raised to prevent stack overflow crashes.

Example: Binary Search Implementation via Recursion

```
def binary_search(arr, target, low, high):
    if low > high:
        return -1  # Base case: element not found

    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search(arr, target, low, mid - 1)
    else:
        return binary_search(arr, target, mid + 1, high)
```

Time complexity: $\mathcal{O}(\log n)$, Space complexity: $\mathcal{O}(\log n)$ due to recursive stack frames.

2.6 Chapter Summary and Review Questions

1. Contrast `break`, `continue`, and `pass` with practical code examples.
2. How does Python resolve variable identifiers under the LEGB hierarchy? Give an example where `nonlocal` is required.
3. Write a recursive function to compute the Greatest Common Divisor (GCD) of two integers using the Euclidean algorithm.
4. Explain the consequence of defining `def register(item, log=[])` and suggest the standard defensive fix.

Chapter 3

Data Structures: Strings, Lists, Tuples, Dictionaries, and Sets

3.1 Sequences: Strings and Lists

Python provides rich sequence types that support common operations: indexing, slicing, concatenation, repetition, and membership testing.

3.1.1 Strings: Immutable Text Sequences

Strings are immutable sequences of Unicode codepoints. Indexing uses zero-based non-negative offsets from left, or negative offsets from the right:

$$\text{slice syntax: } \texttt{s[start:stop:step]} \tag{3.1}$$

When `step` is negative, the slice traverses in reverse (e.g., `s[::-1]` reverses string `s`).

```
payload = "  SENSOR_ID:A49_VAL:104.5_STATUS:OK  "
cleaned = payload.strip()
tokens = cleaned.split('_')
formatted_key = ":".join(tokens[:2])
print("Extracted Tokens:", tokens)
print("Reconstructed:", formatted_key)
```

Listing 3.1: Key String Transformation Methods

3.1.2 Lists: Dynamic Heterogeneous Arrays

Lists are mutable sequences implemented in CPython as dynamic arrays of pointers (`PyObject**`). As elements are appended, the array automatically over-allocates geometric headroom to provide amortized $\mathcal{O}(1)$ append operations.

Operation	Method / Syntax	Description	Time Complexity
Append	`L.append(x)`	Adds element to end	$\mathcal{O}(1)$ amortized
Insert	`L.insert(i, x)`	Shifts elements right of index i	$\mathcal{O}(n)$
Pop End	`L.pop()`	Removes and returns terminal element	$\mathcal{O}(1)$
Pop Index	`L.pop(i)`	Shifts elements left of index i	$\mathcal{O}(n)$
Sort	`L.sort()`	In-place adaptive Timsort	$\mathcal{O}(n \log n)$
Reverse	`L.reverse()`	In-place element reversal	$\mathcal{O}(n)$

Table 3.1: CPython List Core Operations and Asymptotic Complexity.

3.1.3 List Comprehensions

List comprehensions provide a concise syntactic construct to generate lists:

```
# Basic transformation
squares = [x**2 for x in range(10) if x % 2 == 0]

# Flattening 2D matrix
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [val for row in matrix for val in row]
```

Listing 3.2: Comprehension Patterns

3.2 Tuples: Immutable Heterogeneous Records

Tuples are immutable sequence collections created using parentheses `()` or comma-separated literals. Once instantiated, tuple size and element bindings cannot be modified.

Principle: Tuple Hashability and Immutability

A tuple is hashable (and therefore eligible as a dictionary key or set member) if and only if all of its constituent elements are themselves hashable (immutable).

```
# Coordinate packing and unpacking
point3d = (12.5, -4.2, 85.0)
x, y, z = point3d

# Idiomatic variable swap in Python
a, b = 10, 20
a, b = b, a  # Implemented via tuple packing and unpacking
```

Listing 3.3: Tuple Packing, Unpacking, and Swapping

3.3 Dictionaries: Key-Value Hash Maps

Dictionaries (`dict`) implement an associative array mapping unique keys to arbitrary value objects.

3.3.1 CPython Hash Table Architecture

CPython implements compact dictionaries using an internal sparse index array pointing to a dense array of entries containing `(hash, key, value)`.

- **Average Time Complexity:** $\mathcal{O}(1)$ for lookup, insertion, and deletion.
- **Key Requirements:** Keys must implement `__hash__()` and `__eq__()` protocols and remain invariant over time.

```
telemetry = {"temp": 28.4, "pressure": 101.3, "humidity": 65.2}

# Safe access with default fallback
lux = telemetry.get("ambient_lux", 0.0)

# Dictionary comprehension
scaled_telemetry = {k: v * 1.1 for k, v in telemetry.items()}
```

Listing 3.4: Dictionary Manipulation and Frequency Counter

3.4 Sets: Mathematical Collection of Unique Elements

Sets (`set`) are mutable, unordered collections of unique hashable objects, optimized for membership tests ($\mathcal{O}(1)$ average) and mathematical set algebra:

- Union: `A | B` or `A.union(B)`
- Intersection: `A & B` or `A.intersection(B)`
- Difference: `A - B` or `A.difference(B)`
- Symmetric Difference: `A ^ B` or `A.symmetric_difference(B)`

3.5 Chapter Summary and Review Questions

1. Differentiate between shallow copy (`copy.copy`) and deep copy (`copy.deepcopy`) with a nested list example.
2. Why cannot a list be used as a dictionary key, whereas a tuple can?
3. Given `s = "engineering"`, write a single-line dictionary comprehension that counts the frequency of each character.
4. Explain the time complexity differences between checking membership (`x in collection`) for a `list` of size N versus a `set` of size N.

Chapter 4

Scientific Computing with NumPy: Vectorized Arrays

4.1 Motivation: Why NumPy for Engineering?

Standard Python lists store heterogeneous pointers to heap objects, causing pointer indirection overhead, cache misses, and significant memory overhead (typically 8 bytes per pointer plus 28 bytes per integer object). In high-performance engineering simulations, processing millions of numerical values using explicit Python loops is prohibitively slow.

Definition: NumPy N-Dimensional Array (`ndarray`)

A NumPy array is a contiguous, fixed-size memory buffer storing homogeneous C-primitive elements (e.g., 64-bit float, 32-bit int) described by a metadata descriptor:

1. **Data Pointer:** Starting memory address of the raw contiguous buffer.
2. **Shape (`shape`):** A tuple specifying array dimensions $(d_0, d_1, \ldots, d_{k-1})$.
3. **Data Type (`dtype`):** Fixed byte size and interpretation format of each element.
4. **Strides (`strides`):** The number of bytes to step in memory to advance by one index along each axis.

4.2 Array Creation and Structural Inspection

```
import numpy as np

# 1D and 2D arrays
a1 = np.array([1, 2, 3, 4], dtype=np.float64)
a2 = np.zeros((3, 4), dtype=np.int32)
a3 = np.ones((2, 3), dtype=np.float32)
a_identity = np.eye(4)  # 4x4 Identity matrix

# Linear spacing routines
grid_lin = np.linspace(0.0, 10.0, num=101) # Exact step: (stop -
    start)/(num - 1)
grid_range = np.arange(0.0, 10.0, 0.5)       # Step based: excludes
    upper bound

print("Shape   :", a2.shape)     # (3, 4)
print("Dtype   :", a2.dtype)     # int32
print("Strides :", a2.strides)   # (16, 4) bytes
```

Listing 4.1: Fundamental NumPy Creation Routines

4.3 Indexing, Slicing, and Memory Views vs. Copies

NumPy basic slicing produces a **view** into the original array buffer rather than creating a new copy. Modifying a slice directly modifies the original array!

```
arr = np.array([10, 20, 30, 40, 50])
view_slice = arr[1:4]
view_slice[0] = 999
print(arr)  # Output: [10, 999, 30, 40, 50] -> Original array modified!

# Explicit copy to preserve original data
copy_arr = arr[1:4].copy()

# Boolean Masking
mask = arr > 30
filtered = arr[mask]
print("Values > 30:", filtered)
```

Listing 4.2: Views vs. Deep Copies and Fancy Masking

4.4 Vectorization and Universal Functions (ufuncs)

Vectorization replaces explicit Python `for` loops with compiled, low-level C loops executing SIMD (Single Instruction Multiple Data) machine instructions.

NumPy Broadcasting Semantics

Two dimensions are compatible for element-wise broadcasting operations if and only if:

1. They are equal in size ($d_1 = d_2$), or
2. One of the dimensions is exactly 1 ($d_1 = 1$ or $d_2 = 1$).

Trailing axes are aligned first; missing leading dimensions are prepended with size 1.

```
# Signal matrix: shape (100, 3)
signals = np.random.randn(100, 3)

# Calibration offset vector: shape (3,)
offsets = np.array([0.5, -0.2, 1.1])

# Broadcasting automatically expands offsets across the 100 rows
calibrated = signals + offsets  # Shape: (100, 3)
```

Listing 4.3: Broadcasting Demonstration

4.5 Linear Algebra Operations

NumPy's `linalg` submodule links against highly optimized BLAS/LAPACK Fortran libraries (e.g., OpenBLAS, Intel MKL):

```
# Mesh equations:
# 10*I1 -  2*I2 -  4*I3 = 12
# -2*I1 + 15*I2 -  3*I3 = 0
# -4*I1 -  3*I2 + 12*I3 = -6

A = np.array([[ 10.0,  -2.0,  -4.0],
```

```
                [ -2.0,   15.0,   -3.0],
                [ -4.0,   -3.0,   12.0]])

b = np.array([12.0, 0.0, -6.0])

# Direct inversion vs solve
# Preferred: np.linalg.solve (LU decomposition with partial pivoting)
I = np.linalg.solve(A, b)
det_A = np.linalg.det(A)
eigenvalues, eigenvectors = np.linalg.eig(A)

print("Mesh Currents (I1, I2, I3) in Amperes:", I)
print("Determinant of Resistance Matrix     :", det_A)
```

Listing 4.4: Solving Linear Systems $Ax = b$ in Circuit Analysis

4.6 Chapter Summary and Review Questions

1. Contrast the internal memory architecture of a Python list with a NumPy `ndarray`.
2. State the formal broadcasting rules. Given array A of shape $(4, 1, 8)$ and array B of shape $(5, 8)$, determine whether they are broadcast-compatible and calculate the resulting shape.
3. Why is `np.linalg.solve(A, b)` numerically preferred over computing `np.linalg.inv(A) @ b`?
4. Explain the difference between basic slicing and fancy indexing in terms of memory views versus copies.

Chapter 5

Data Analysis with Pandas and Visualisation with Matplotlib

5.1 Introduction to Tabular Data Analysis with Pandas

Pandas is the premier Python library for structured data manipulation. Built atop NumPy, Pandas introduces labeled, heterogeneous tabular structures that support relational database operations, time series analysis, and statistical processing.

5.1.1 Primary Data Structures: Series and DataFrame

1. **Series:** A 1-dimensional labeled homogeneous array capable of holding any data type. It consists of an underlying data array and an associated **Index**.
2. **DataFrame:** A 2-dimensional tabular structure with labeled axes (rows and columns). It can be viewed as an ordered dictionary of `Series` sharing a common index.

```
import pandas as pd
import numpy as np

# Constructing DataFrame from dictionary of lists
data = {
    'Timestamp': pd.date_range(start='2026-01-01', periods=5, freq='h'),
    'Node_ID': ['N1', 'N2', 'N1', 'N3', 'N2'],
    'Voltage_V': [3.31, 4.98, 3.28, np.nan, 5.02],
    'Current_mA': [12.4, 45.1, 11.9, 8.5, 46.0]
}
df = pd.DataFrame(data)
print(df.info())
```

Listing 5.1: Creating and Manipulating DataFrames

5.1.2 Selection and Indexing: `.loc[]` vs. `.iloc[]`

- **`.loc[]`: Label-based indexing.** Slicing with `.loc` includes **both** the start and the stop endpoints!
- **`.iloc[]`: Integer-position-based indexing.** Slicing follows standard Python sequence semantics (half-open interval $[start, stop)$, excluding the stop boundary).

```
# Select rows 0 to 2 and specific columns by label
sub_loc = df.loc[0:2, ['Node_ID', 'Voltage_V']]

# Select rows 0 to 2 (indices 0 and 1) and columns 1 to 3 by position
sub_iloc = df.iloc[0:2, 1:3]
```

Listing 5.2: Label vs Position Slicing

5.1.3 Data Cleaning: Handling Missing Values

Real-world engineering sensor streams contain missing data resulting from packet drops or sensor failures:

```
# Check missing counts
print(df.isna().sum())

# Drop rows with any NaN
df_clean = df.dropna()

# Impute missing values with group mean
df['Voltage_V'] = df['Voltage_V'].fillna(df['Voltage_V'].mean())
```

Listing 5.3: Handling Missing Data

5.1.4 Aggregation and Grouping: Split-Apply-Combine

```
summary = df.groupby('Node_ID').agg({
    'Voltage_V': ['mean', 'std'],
    'Current_mA': ['mean', 'max']
})
print(summary)
```

Listing 5.4: Group-by Aggregations

5.2 Engineering Data Visualisation with Matplotlib

Matplotlib is a multi-platform data visualization library built on NumPy arrays. It provides full control over every graphic primitive on the canvas.

5.2.1 Matplotlib Object-Oriented Hierarchy

While the `pyplot` interface mimics MATLAB procedural commands, the object-oriented API provides robust control:

- **Figure:** The top-level container holding all canvas elements, subplots, titles, and legends.
- **Axes:** The actual plotting region bounding the data coordinate system with x/y scales, ticks, and labels.

```
import matplotlib.pyplot as plt
import numpy as np

# Synthetic sensor telemetry
t = np.linspace(0.0, 2.0, 500)
freq = 5.0
signal_clean = 3.3 * np.sin(2 * np.pi * freq * t)
noise = 0.4 * np.random.randn(len(t))
signal_noisy = signal_clean + noise

# Create Figure and Axes
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

# Subplot 1: Time domain waveform
ax1.plot(t, signal_noisy, color='gray', alpha=0.6, label='Noisy ADC
    Signal')
ax1.plot(t, signal_clean, color='crimson', lw=2, label='True
    Fundamental (5 Hz)')
```

```
ax1.set_ylabel('Voltage (V)', fontsize=11)
ax1.set_title('Sensor Telemetry Acquisition: Time Domain Waveform',
    fontsize=12, fontweight='bold')
ax1.grid(True, linestyle='--', alpha=0.7)
ax1.legend(loc='upper right')

# Subplot 2: Error residual distribution
residual = signal_noisy - signal_clean
ax2.plot(t, residual, color='teal', lw=1.2, label='Noise Residual')
ax2.axhline(0, color='black', linestyle=':', lw=1)
ax2.set_xlabel('Time (s)', fontsize=11)
ax2.set_ylabel('Residual (V)', fontsize=11)
ax2.grid(True, linestyle='--', alpha=0.7)
ax2.legend(loc='upper right')

plt.tight_layout()
# Save high-resolution vector graphic
# plt.savefig('sensor_telemetry.pdf', dpi=300)
```

Listing 5.5: Engineering Signal Visualization using the OO API

5.3 Standard Statistical Plot Types

1. **Line Plot (`ax.plot`):** Displays continuous time-series trajectories.
2. **Scatter Plot (`ax.scatter`):** Visualizes bi-variate correlation and clustering.
3. **Histogram (`ax.hist`):** Approximates continuous probability density functions.
4. **Bar Chart (`ax.bar`):** Compares discrete categorical quantities.
5. **Box Plot (`ax.boxplot`):** Visualizes 5-number summary (minimum, Q1, median, Q3, maximum) and outlier anomalies.

5.4 Chapter Summary and Review Questions

1. Compare and contrast the endpoint indexing behavior of `.loc[]` versus `.iloc[]`.
2. Explain the three stages of the Split-Apply-Combine strategy implemented by Pandas `groupby()`.
3. Why is the Object-Oriented API (`fig, ax = plt.subplots()`) preferred over procedural state-machine calls (`plt.plot()`) in professional engineering workflows?
4. Given a DataFrame `df` with missing values in column `'temperature'`, write code to replace missing entries with the rolling median of the preceding 3 observations.

Chapter 6

Real-World Engineering Applications: OOP, File I/O, and Exceptions

6.1 Object-Oriented Programming (OOP) in Python

Object-Oriented Programming models real-world hardware systems as computational objects combining internal state (attributes) with behaviors (methods).

6.1.1 Classes, Instances, and the `self` Parameter

In Python, a class is a blueprint for instantiating objects. The special method `__init__` serves as the object constructor. The explicit `self` parameter refers to the current instance invoking the method.

```
class SensorNode:
    # Class-level attribute shared across all instances
    BUS_STANDARD = "I2C"

    def __init__(self, node_id: str, address: int, v_ref: float = 3.3):
        # Instance attributes
        self.node_id = node_id
        self.address = address
        self.v_ref = v_ref
        self._calibration_offset = 0.0  # Protected attribute by
            convention

    def read_raw_adc(self) -> int:
        """Simulate reading 10-bit ADC integer."""
        import random
        return random.randint(0, 1023)

    def get_voltage(self) -> float:
        raw = self.read_raw_adc()
        return (raw / 1023.0) * self.v_ref + self._calibration_offset

    def set_offset(self, offset: float):
        if abs(offset) > 0.5:
            raise ValueError("Calibration offset exceeds safe +/- 0.5V
                limit.")
        self._calibration_offset = offset
```

Listing 6.1: Modeling an Embedded Analog Sensor Class

6.1.2 Inheritance and Polymorphism

Subclasses inherit attributes and methods from base classes and can override behavior using the `super()` proxy object:

```
class TemperatureSensor(SensorNode):
```

```
    def __init__(self, node_id: str, address: int, sensitivity_mv_c:
        float = 10.0):
        super().__init__(node_id, address, v_ref=3.3)
        self.sensitivity = sensitivity_mv_c / 1000.0  # Convert to
            V/deg C

    def get_temperature_celsius(self) -> float:
        voltage = self.get_voltage()
        # LM35 characteristic: 10 mV / deg C
        return voltage / self.sensitivity
```

Listing 6.2: Specialized Temperature Sensor Subclass

6.2 Defensive File Handling and Context Managers

6.2.1 The `with` Statement (Context Manager Protocol)

Engineering scripts must guarantee that operating system file descriptors and hardware communication sockets are promptly closed, even if runtime errors occur. The `with` statement enforces the Context Manager protocol (`__enter__` and `__exit__`).

```
import csv

# Writing structured engineering log data
log_file = "sensor_telemetry.csv"
records = [
    {"timestamp": "12:00:01", "node": "TMP_01", "temp": 24.5},
    {"timestamp": "12:00:02", "node": "TMP_01", "temp": 24.8},
    {"timestamp": "12:00:03", "node": "TMP_01", "temp": 25.1}
]

with open(log_file, mode="w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["timestamp", "node", "temp"])
    writer.writeheader()
    writer.writerows(records)

# Safe reading
with open(log_file, mode="r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"Time: {row['timestamp']} | Temp:
            {float(row['temp']):.1f} C")
```

Listing 6.3: Reading and Writing Telemetry Logs with CSV

6.3 Robust Exception Handling

Exceptions provide a structured mechanism for handling runtime errors without crashing mission-critical processes.

Definition: Exception Architecture

The `try-except-else-finally` suite executes instructions with deterministic cleanup semantics:

- **`try`:** Encloses risky code that may raise an exception.

- **except ExceptionType as err:** Catches and handles specific exception instances.
- **else:** Executes if and only if no exception was raised in the `try` block.
- **finally:** Executes unconditionally under all circumstances (even after `return` or uncaught exceptions), ideal for releasing hardware locks.

```
def parse_telemetry_packet(raw_string: str) -> dict:
    try:
        parts = raw_string.strip().split(',')
        if len(parts) < 3:
            raise IndexError("Packet contains fewer fields than
                expected format.")

        node_id = parts[0]
        voltage = float(parts[1])
        status_code = int(parts[2])

    except ValueError as ve:
        print(f"Data corruption error: numeric conversion failed: {ve}")
        return {}
    except IndexError as ie:
        print(f"Malformed framing error: {ie}")
        return {}
    else:
        print("Packet parsed successfully with zero errors.")
        return {"node": node_id, "voltage": voltage, "status":
            status_code}
    finally:
        # Guarantee watchdog timer reset
        pass
```

Listing 6.4: Comprehensive Exception Handling Pattern

6.4 Real-World Case Study: Automated Diagnostic Log Processing

Modern electronics systems deploy autonomous Python scripts to parse daily hardware crash dumps, calculate mean-time-between-failures (MTBF), generate analytical figures with Matplotlib, and dispatch diagnostic summaries to engineering teams.

6.5 Chapter Summary and Review Questions

1. Explain how Python's `with open(...) as f:` guarantees file closure even if an unhandled exception occurs inside the block.
2. Contrast class attributes versus instance attributes. In what scenarios is a class attribute appropriate?
3. What is the purpose of the `else` and `finally` clauses in a `try-except` construct?
4. Design an object-oriented Python hierarchy representing electronic lab instruments (`PowerSupply`, `Oscilloscope`, `Multimeter`) derived from an abstract base class `LabInstrument`.